

Class 1

ال Class هو التصميم اللي بنيني منه Objects.

```
class Car {  
    String color;  
    String brand;  
}
```

هنا Car هو ال Class, وفيه بيانات العربية.

Object 2

ال Object هو نسخة حقيقية من ال Class.

```
Car car1 = new Car();
```

يعني عملنا عربية فعلية من قالب Car.

مممكن نعمل أكثر من Object:

```
Car car1 = new Car();  
Car car2 = new Car();
```

وكل Object له بياناته الخاصة.

Attributes / Fields 3

هي البيانات الموجودة داخل ال Class.

```
class Car {  
    String color;  
    String brand;  
    int year;  
}
```

ال Method هي وظيفة ال Object يقدر يعملها.

```
class Car {  
    void start() {  
        System.out.println("Car Started");  
    }  
}
```

وبعدين:

```
Car car = new Car();  
car.start();
```

Constructor 5

ال Constructor بيتنفذ تلقائيًا لما تعمل Object.

```
class Car {  
    Car() {  
        System.out.println("Car Created");  
    }  
}
```

لما تعمل:

```
Car car = new Car();
```

هيطبع:

```
Car Created
```

:Note

- اسمه نفس اسم ال Class.
- مفيش Return Type.
- بيتنفذ عند إنشاء Object.

Constructor Parameters 6

يمكن Constructor استقبال بيانات.

```
class Car {  
    String color;  
  
    Car(String c) {  
        color = c;  
    }  
}
```

ولما تعمل:

```
Car car = new Car("Red");
```

تصبح:

```
color = Red
```

Constructor Overloading 7

يمكن يكون عندك أكثر من Constructor، بشرط إن ال Parameters تختلف.

```
Car() {  
}  
  
Car(String color) {  
}  
  
Car(String color, int year) {  
}
```

this معناها: ال Object الحالي.

مثلاً:

```
class Car {  
    String color;  
  
    Car(String color) {  
        this.color = color;  
    }  
}
```

هنا عندنا color مرتين:

this.color

ده ال Field الموجود في Object.

color

ده ال Parameter.

ف:

this.color = color;

معناها: حط قيمة ال Parameter في Field بتاع ال Object.

Access Modifiers 9

Modifier	نفس الـ Class	نفس الـ Package	Child في Package مختلفة	أي Class في أي مكان
public	✓	✓	✓	✓
protected	✓	✓	*✓	✗
default	✓	✓	✗	✗
private	✓	✗	✗	✗

Encapsulation 10

معناها إننا نحمي البيانات ونخلي الوصول ليها من خلال Methods.

مثلاً:

```
class BankAccount {  
    private double balance;  
  
    public double getBalance() {  
        return balance;  
    }  
  
    public void setBalance(double balance) {  
        this.balance = balance;  
    }  
}
```

هنا:

getBalance()

تجيب القيمة.

و:

setBalance()

تغير القيمة.

وده بيمنع التعديل العشوائي على البيانات.

Getter 11

بيستخدم عشان يجيب قيمة متغير private.

```
public int getAge() {  
    return age;  
}
```

Setter 12

يستخدم عشان يغير قيمة متغير private.

```
public void setAge(int age) {  
    this.age = age;  
}
```

static 13

static معناها إن الحاجة مرتبطة بال Class نفسه مش بكل Object لوحده.

مثلاً:

```
class Student {  
    static String school = "Modern Academy";  
}
```

تقدر تستخدمها:

```
System.out.println(Student.school);
```

مش لازم تعمل Object.

Inheritance 14

ال Inheritance يعني Class يرث من Class ثاني.

```
class Animal {  
    void eat() {  
        System.out.println("Eating");  
    }  
}  
  
class Dog extends Animal {  
  
}
```

ال Dog ورث eat () من Animal.

فممکن:

```
Dog dog = new Dog();  
dog.eat();
```

Child و Parent 15

في:

```
class Dog extends Animal
```

يبقى:

```
Animal = Parent  
Dog = Child
```

ال Child بياخذ الخصائص وال Methods اللي يمكنه وراثتها من ال Parent.

super معناها: الأب / Parent Class.

مثلاً لو عندك:

```
class Animal {  
    String name = "Animal";  
}  
  
class Dog extends Animal {  
    String name = "Dog";  
}
```

لو كتبت:

```
System.out.println(super.name);
```

هتجيب:

```
Animal
```

يعني المتغير الموجود في الـ Parent.

كمان:

```
super.method();
```

تشغل Method من الـ Parent.

و:

```
super();
```

تستدعي الـ Constructor Parent.

Method Overriding 17

ال Child يعمل نفس ال Method الموجودة في ال Parent، لكن بتطبيق مختلف.

```
class Animal {  
    void sound() {  
        System.out.println("Animal Sound");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Woof");  
    }  
}
```

لما تعمل:

```
Dog dog = new Dog();  
dog.sound();
```

النتيجة:

Woof

لأن Dog عمل Override لل Method.

Method Overloading 18

نفس اسم ال Method، لكن Parameters مختلفة.

```
int add(int a, int b) {  
    return a + b;  
}  
  
int add(int a, int b, int c) {  
    return a + b + c;  
}
```

نفس الاسم:

add

لكن عدد ال Parameters مختلف.

Polymorphism 19

معناها ببساطة: نفس الأمر ممكن يتصرف بطريقة مختلفة حسب ال Object.

مثلاً:

```
Animal animal = new Dog();  
animal.sound();
```

رغم إن نوع ال Reference هو Animal، ال Object الحقيقي هو Dog.

فهيشغل:

```
Dog.sound()
```

معناها إنك تظهر للمستخدم المهم فقط وتخفي تفاصيل التنفيذ.

مثلاً أنت بتستخدم العربية:

دوس بنزين → العربية تتحرك

مش محتاج تعرف تفاصيل عمل المحرك.

في Java عندنا:

```
abstract class Animal {  
    abstract void sound();  
}
```

ال Implementation مفياش abstract method.

ال Child هو اللي يحددها.

Abstract Class 21

Class عليه كلمة:

```
abstract
```

مثلاً:

```
abstract class Animal {  
    abstract void sound();  
}
```

مينفعش تعمل منه Object مباشرة:

```
Animal a = new Animal(); // Error
```

لكن تقدر تعمل Child:

```
class Dog extends Animal {  
    void sound() {  
        System.out.println("Woof");  
    }  
}
```

Interface 22

ال Interface عبارة عن عقد / Contract.
بتقول لل Class: لازم تعمل ال Methods دي.
مثلاً:

```
interface Animal {  
    void sound();  
}
```

وبعدين:

```
class Dog implements Animal {  
    public void sound() {  
        System.out.println("Woof");  
    }  
}
```

هنا Dog لازم يعمل sound().

implements 23

بنستخدمها لما Class يطبق Interface.

```
class Dog implements Animal
```

يعني: Dog ملتزم بال methods الموجودة في Animal Interface.